

Practical - 8

1. Memory Allocation

```
harshika@siri:~$ gcc -Wall -g memory_allocation.c -o memory_allocation
harshika@siri:~$ ./memory_allocation
=====
DYNAMIC MEMORY ALLOCATION
=====
1. malloc() DEMONSTRATION
-----
Memory allocated using malloc().
Values stored using malloc:
10 20 30 40 50
2. calloc() DEMONSTRATION
-----
Memory allocated using calloc().
Initial values are:
0 0 0 0 0
3. realloc() DEMONSTRATION
-----
Memory resized using realloc().
Values after realloc:
10 20 30 40 50 60 70 80 90 100
4. free() DEMONSTRATION
-----
malloc/realloc memory released using free().
calloc memory released using free().
=====
All allocated memory has been released.
=====
harshika@siri:~$ gcc -Wall -g memory_allocation.c -o memory_allocation
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <string.h>
4
5int main()
6{
7    printf("===== \n");
8    printf(" DYNAMIC MEMORY ALLOCATION \n");
9    printf("===== \n");
10
11    /* -----
12     1. MALLOC ----- */
13
14    printf("\n1. malloc() DEMONSTRATION \n");
15    printf("----- \n");
16
17    int *arr;
18
19    arr = (int *)malloc(5 * sizeof(int));
20
21    if (arr == NULL)
22    {
23        printf("malloc failed \n");
24        return 1;
25    }
26
27    printf("Memory allocated using malloc \n");
28
29    for (int i = 0; i < 5; i++)
30    {
31        arr[i] = (i + 1) * 10;
32    }
33
34    printf("Values stored using malloc \n");
35
36    for (int i = 0; i < 5; i++)
37    {
38        printf("%d ", arr[i]);
39    }
40
41    printf("\n");
42
43    /* -----
44     2. CALLOC ----- */
45
46    printf("\n2. calloc() DEMONSTRATION \n");
47    printf("----- \n");
48
49    int *zero_arr;
50
51    zero_arr = (int *)calloc(5, sizeof(int));
52
53    printf("Memory allocated using calloc \n");
54    printf("Initial values are: \n");
55    for (int i = 0; i < 5; i++)
56    {
57        zero_arr[i] = 0;
58    }
59    printf("Values stored using calloc \n");
60    for (int i = 0; i < 5; i++)
61    {
62        printf("%d ", zero_arr[i]);
63    }
64    printf("\n");
65
66    /* -----
67     3. REALLOC ----- */
68
69    printf("\n3. realloc() DEMONSTRATION \n");
70    printf("----- \n");
71
72    int *arr2;
73    arr2 = (int *)realloc(arr, 10 * sizeof(int));
74
75    if (arr2 == NULL)
76    {
77        printf("realloc failed \n");
78        return 1;
79    }
80
81    printf("Memory resized using realloc \n");
82
83    for (int i = 0; i < 10; i++)
84    {
85        arr2[i] = (i + 1) * 10;
86    }
87
88    printf("Values stored using realloc \n");
89    for (int i = 0; i < 10; i++)
90    {
91        printf("%d ", arr2[i]);
92    }
93    printf("\n");
94
95    /* -----
96     4. FREE ----- */
97
98    printf("\n4. free() DEMONSTRATION \n");
99    printf("----- \n");
100
101    free(arr);
102    free(arr2);
103
104    printf("All allocated memory has been released. \n");
105    printf("===== \n");
106}
```

2. Copy on Write using Fork()

```
harshika@siri:~$ gcc -Wall -g copy_on_write.c -o copy_on_write
harshika@siri:~$ ./copy_on_write
=====
COPY-ON-WRITE DEMONSTRATION
=====
Parent PID: 5633
Memory initialized in parent.
[PARENT - BEFORE fork() - PID 5633]
VmSize: 13088 KB
VmRSS: 12056 KB
Creating child process using fork()...
=====
PARENT PROCESS
=====
Parent PID: 5633
Child PID: 5634
=====
CHILD PROCESS
=====
Child PID: 5634
[PCHILD - AFTER fork(), BEFORE modification - PID 5634]
VmSize: 13088 KB
VmRSS: 11240 KB
Child is modifying the shared memory...
Child modification completed.
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <unistd.h>
4#include <sys/types.h>
5#include <sys/wait.h>
6#include <string.h>
7
8#define SIZE (10 * 1024 * 1024)
9
10/*
11 * Global array.
12 */
13/* It is large enough to make the
14 * Copy-on-Write behavior easier to observe.
15 */
16char data[SIZE];
17
18/* -----
19 * Function to display memory information
20 * ----- */
21void show_memory(const char *process_name)
22{
23    FILE *file = fopen("/proc/self/status", "r");
24    if (file == NULL)
25    {
26        perror("fopen");
27        return;
28    }
29    printf("\n[%s - PID %d] \n", process_name, getpid());
30    while (fgetc(line, sizeof(line), file))
31    {
32        if (strncmp(line, "VmSize:", 7) == 0 ||
33            strncmp(line, "VmRSS:", 6) == 0)
34        {
35            printf("%s", line);
36        }
37    }
38    fclose(file);
39}
40
41int main()
42{
43    printf("===== \n");
44    printf("COPY-ON-WRITE DEMONSTRATION \n");
45    printf("===== \n");
46
47    /* Parent process */
48    show_memory("PARENT PROCESS");
49
50    /* Child process */
51    pid_t pid = fork();
52    if (pid == 0)
53    {
54        /* Child process */
55        show_memory("CHILD PROCESS");
56        /* Modify shared memory */
57        for (int i = 0; i < 1024; i++)
58        {
59            data[i] = i;
60        }
61        printf("Child is modifying the shared memory...\n");
62        printf("Child modification completed.\n");
63    }
64    else if (pid > 0)
65    {
66        /* Parent process */
67        show_memory("PARENT PROCESS");
68        wait(&pid);
69    }
70    else
71    {
72        perror("fork");
73        return 1;
74    }
75}
```